UNIT III

Dictionaries, Tuples & Sets

Dictionary operations & methods · Tuples · Tuple–list–dictionary relations · zip() · Sets · Set methods · Frozenset · Sample Programs 13–17

➤ What you will learn in this unit

- Storing data as **key : value** pairs in **dictionaries** and using their functions, methods and **del**.
- Creating **tuples**, their operations, and how they relate to lists and dictionaries; the **zip()** function.
- Working with **sets** (unique items, mathematical operations) and **frozenset**.
- Solutions of the sample programs **13 to 17** with real output.

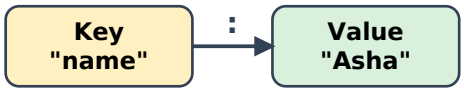
PART A — DICTIONARIES

1. Creating a dictionary

A **dictionary** stores data as **key : value** pairs inside curly braces. Keys must be **unique and immutable** (str, int, tuple); values can be anything. Since Python 3.7 a dictionary remembers the **insertion order**.

student (dictionary)

```
"name"   :  "Asha"
 "age"   :  18
"city"   :  "Kadapa"
```



one key-value pair

Access a value with its KEY: student["name"]

```
student = {"name": "Asha", "age": 18, "city": "Kadapa"}   # using { }
empty = {}                                                # empty dictionary
d2 = dict(a=1, b=2)                                       # dict() function
d3 = dict([("x", 10), ("y", 20)])                         # from list of pairs
print(student)
print(d2, d3, type(empty))
```

Output

```
{'name': 'Asha', 'age': 18, 'city': 'Kadapa'}
{'a': 1, 'b': 2} {'x': 10, 'y': 20} <class 'dict'>
```

2. Accessing and modifying key : value pairs

Task	Code	Note
Access value	`d["name"]`	KeyError if key is missing
Safe access	`d.get("phone", "NA")`	Returns default (None) if missing — no error
Modify value	`d["age"] = 19`	Key exists → value is updated
Add new pair	`d["email"] = "a@x.com"`	Key does not exist → new pair added
Check key	`"age" in d`	Membership tests the **keys**
Loop	`for k in d: / d.items()`	Loop over keys / (key, value) pairs

```
d = {"name": "Asha", "age": 18}
print(d["name"], d.get("phone", "NA"))
d["age"] = 19                       # modify
d["city"] = "Kadapa"                # add
print(d)
for k, v in d.items():              # iterate over pairs
    print(k, "->", v)
```

Output

```
Asha NA
{'name': 'Asha', 'age': 19, 'city': 'Kadapa'}
name -> Asha
age -> 19
city -> Kadapa
```

3. Built-in functions and dictionary methods

Built-in function	Result
`len(d)`	Number of key-value pairs
`sorted(d)`	List of keys in sorted order
`max(d) min(d)`	Largest / smallest **key**
`sum(d)`	Sum of the keys (if numeric)
`list(d) tuple(d)`	Keys as a list / tuple
`any(d) all(d)`	Truth test on keys

Method	Purpose
`keys() values() items()`	View of all keys / all values / all (key, value) pairs
`get(key, default)`	Value for key or default if absent
`update(other)`	Add / overwrite pairs from another dictionary
`pop(key)`	Remove key and **return** its value
`popitem()`	Remove and return the **last inserted** pair
`setdefault(key, v)`	Return value if key exists, otherwise add key with v
`copy() clear()`	Shallow copy / remove all items
`dict.fromkeys(keys, v)`	New dictionary with the given keys, all with value v

```
d = {"a": 1, "b": 2, "c": 3}
print(len(d), sorted(d), max(d), sum(d.values()))
```

```
print(list(d.keys()), list(d.values()), list(d.items()))
d.update({"d": 4, "a": 10})
print(d)
print(d.pop("b"), d.popitem(), d)
print(dict.fromkeys(["x", "y"], 0))
```

Output

```
3 ['a', 'b', 'c'] c 6
['a', 'b', 'c'] [1, 2, 3] [('a', 1), ('b', 2), ('c', 3)]
{'a': 10, 'b': 2, 'c': 3, 'd': 4}
2 ('d', 4) {'a': 10, 'c': 3}
{'x': 0, 'y': 0}
```

The del statement

```
d = {"name": "Asha", "age": 18, "city": "Kadapa"}
del d["city"]                # delete one pair by key
print(d)
d.clear()                    # remove all pairs
print(d)
del d                        # delete the dictionary variable itself
```

Output

```
{'name': 'Asha', 'age': 18}
{}
```

i Remember

- **pop()** returns the removed value, **del** does not. Deleting a missing key gives *KeyError*.

PART B — TUPLES

4. Creating tuples and basic operations

A **tuple** is an ordered collection like a list, but it is **immutable** — items cannot be changed, added or removed after creation. It is written in round brackets. Tuples are faster than lists and safe for fixed data.

```
t1 = (10, 20, 30)
t2 = 5, 6, 7                    # brackets are optional
single = (99,)                  # one-item tuple NEEDS a comma
t3 = tuple([1, 2, 3])           # tuple() function: list -> tuple
t4 = tuple("abc")               # string -> tuple
print(t1, t2, single, t3, t4)
print(type((99)), type((99,)))
```

Output

```
(10, 20, 30) (5, 6, 7) (99,) (1, 2, 3) ('a', 'b', 'c')
<class 'int'> <class 'tuple'>
```

Operation	Example	Result
Concatenation	`(1,2) + (3,4)`	(1, 2, 3, 4)
Repetition	`(1,2) * 2`	(1, 2, 1, 2)
Membership	`3 in (1,2,3)`	True
Length	`len((1,2,3))`	3
Unpacking	`a, b = (5, 6)`	a = 5, b = 6
Change an item	`t[0] = 5`	**TypeError** (immutable)

5. Indexing, slicing and built-in functions on tuples

Indexing and slicing are the same as for lists and strings: *t[0]*, *t[-1]*, *t[1:3]*, *t[::-1]*.

Function / method	Result
`len(t) max(t) min(t) sum(t)`	Size, largest, smallest, total
`sorted(t)`	Sorted **list** of the items (tuple itself is not changed)
`t.count(x)`	Number of times x occurs
`t.index(x)`	Index of the first occurrence of x
`tuple(seq)`	Convert a list / string / range into a tuple

```
t = (4, 8, 15, 8, 23, 42)
print(t[0], t[-1], t[1:4], t[::-1])
print(len(t), max(t), min(t), sum(t))
print(t.count(8), t.index(15), sorted(t))
```

Output

```
4 42 (8, 15, 8) (42, 23, 8, 15, 8, 4)
6 42 4 100
2 2 [4, 8, 8, 15, 23, 42]
```

6. Relation between tuples and lists

	List	Tuple
Brackets	`[ ]`	`( )`
Mutable?	**Yes** — can add, remove, change	**No** — fixed once created
Methods	Many (append, remove, sort …)	Only count() and index()
Speed / memory	Slower, more memory	Faster, less memory
Use for	Data that changes	Fixed data (coordinates, dates), dictionary keys

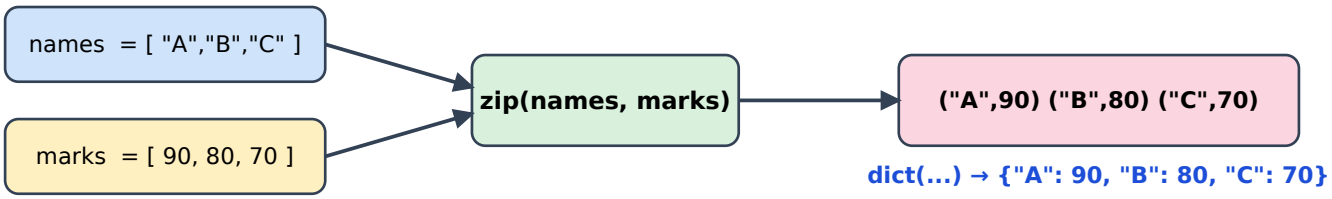
```
L = [1, 2, 3]
T = tuple(L)              # list -> tuple
print(T)
L2 = list(T)              # tuple -> list
L2.append(4)              # now it can change
print(L2)
a, b = 5, 10
a, b = b, a               # swapping uses tuple packing / unpacking
print(a, b)
```

Output

```
(1, 2, 3)
[1, 2, 3, 4]
10 5
```

7. Relation between tuples and dictionaries; the zip() function

- **Tuples as keys**: a tuple is immutable (hashable) so it can be a dictionary key; a list cannot.
- **items()** returns the dictionary as a sequence of **(key, value) tuples**; `dict()` converts a sequence of tuples back into a dictionary.
- **zip(a, b, …)** pairs up items of several sequences into **tuples** — handy to build a dictionary from two lists.




```
names = ["Asha", "Ravi", "Kiran"]
marks = [90, 80, 70]
pairs = list(zip(names, marks))             # list of tuples
print(pairs)
d = dict(zip(names, marks))                 # two lists -> dictionary
print(d)
print(list(d.items()))                      # dictionary -> list of tuples
loc = {(17.38, 78.48): "Hyderabad"}         # tuple as a key
print(loc[(17.38, 78.48)])
for n, m in zip(names, marks):              # loop over two lists together
    print(n, m)
```

Output

```
[('Asha', 90), ('Ravi', 80), ('Kiran', 70)]
{'Asha': 90, 'Ravi': 80, 'Kiran': 70}
[('Asha', 90), ('Ravi', 80), ('Kiran', 70)]
Hyderabad
Asha 90
Ravi 80
Kiran 70
```

PART C — SETS

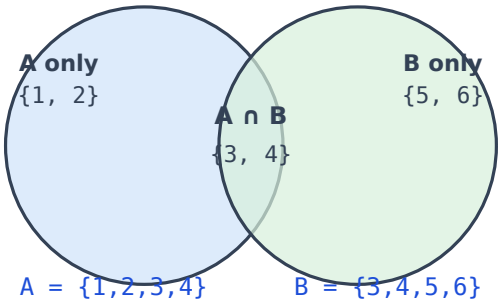
8. Creating sets and set methods

A **set** is an **unordered** collection of **unique** items written in curly braces. No indexing, no duplicates; the set itself is mutable but its items must be immutable.

```
s = {5, 1, 3, 3, 1}                 # duplicates are removed automatically
print(s, len(s))
e = set()                           # empty set  ({} would create a dictionary!)
t = set([1, 2, 2, 3])               # set() from a list - quick way to remove duplicates
print(e, t, type({}))
print(3 in s)
```

Output

```
{1, 3, 5} 3
set() {1, 2, 3} <class 'dict'>
True
```




```
A | B   union          → {1,2,3,4,5,6}
A & B   intersection   → {3,4}
A - B   difference     → {1,2}
A ^ B   symmetric diff → {1,2,5,6}
```

Method	Operator	Purpose
`add(x)`		Add one item
`update(iterable)`		Add many items
`remove(x)`		Delete x — **KeyError** if absent
`discard(x)`		Delete x — no error if absent
`pop()` `clear()`		Remove an arbitrary item / everything
`union(B)`	`A | B`	All items of both sets
`intersection(B)`	`A & B`	Items common to both
`difference(B)`	`A - B`	Items in A but not in B
`symmetric_difference(B)`	`A ^ B`	Items in either set but not in both
`issubset(B)` `issuperset(B)` `isdisjoint(B)`	`A <= B`	Subset / superset / no common items

```
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
print(A | B, A & B, A - B, A ^ B)
print(A.union(B) == A | B)
A.add(10); A.discard(1); A.remove(2)
print(A)
print({3, 4}.issubset(B), A.isdisjoint({100}))
```

Output

```
{1, 2, 3, 4, 5, 6} {3, 4} {1, 2} {1, 2, 5, 6}
True
{3, 4, 10}
True True
```

9. Frozenset

A **frozenset** is an **immutable** set. It supports all read-only set operations (union, intersection ...) but not add / remove. Because it is hashable it can be used as a **dictionary key** or as an item of another set.

```
f = frozenset([1, 2, 3, 2])
print(f, type(f))
print(f | {4}, 2 in f)
try:
    f.add(5)
except AttributeError as e:
    print("Error:", e)
d = {frozenset([1, 2]): "pair"}       # frozenset used as dictionary key
print(d[frozenset([2, 1])])
```

Output

```
frozenset({1, 2, 3}) <class 'frozenset'>
frozenset({1, 2, 3, 4}) True
Error: 'frozenset' object has no attribute 'add'
pair
```

	List	Tuple	Set	Dictionary
Brackets	`[ ]`	`( )`	`{ }`	`{k: v}`
Ordered	Yes	Yes	No	Yes (insertion order)
Mutable	Yes	No	Yes	Yes
Duplicates	Allowed	Allowed	Not allowed	Keys unique
Indexing	By position	By position	No	By key

★ Study Tip

- An **empty {}** is a dictionary — use **set()** for an empty set, and **(x,)** with a comma for a one-item tuple.
- Set operators: **|** union, **&** intersection, **-** difference, **^** symmetric difference.

✓ Quick Revision

- **Dictionary**: {key: value}; keys unique & immutable; get(), keys(), values(), items(), update(), pop(), popitem(); del d[key].
- **Tuple**: immutable, ( ); count() and index() only; one-item tuple needs a comma; tuple ↔ list conversion.
- **zip()** pairs items → dict(zip(keys, values)). **items()** gives (key, value) tuples.
- **Set**: unique, unordered, no indexing; | & - ^; add / remove / discard; **frozenset** = immutable set.

SAMPLE PROGRAMS 13 – 17 (with output)

Program 13

Question

- Write a program to create tuples (name, age, address, college) for at least two members and concatenate the tuples.

Program

```
member1 = ("Asha", 18, "Kadapa", "JNTUA College")      # (name, age, address, college)
member2 = ("Ravi", 19, "Tirupati", "JNTUA College")
member3 = ("Kiran", 18, "Anantapur", "JNTUA College")

print("Member 1:", member1)
print("Member 2:", member2)

combined = member1 + member2                           # concatenation of tuples
print("\nConcatenated tuple (2 members):\n", combined)

all_members = member1 + member2 + member3
print("\nConcatenated tuple (3 members):\n", all_members)
print("Total items:", len(all_members))
```

Output

```
Member 1: ('Asha', 18, 'Kadapa', 'JNTUA College')
Member 2: ('Ravi', 19, 'Tirupati', 'JNTUA College')

Concatenated tuple (2 members):
 ('Asha', 18, 'Kadapa', 'JNTUA College', 'Ravi', 19, 'Tirupati', 'JNTUA College')

Concatenated tuple (3 members):
 ('Asha', 18, 'Kadapa', 'JNTUA College', 'Ravi', 19, 'Tirupati', 'JNTUA College', 'Kiran', 18, 'Anantapur', 'JN
Total items: 12
```

i Remember

- Tuples are immutable, so **+** does not change the old tuples — it creates a **new** tuple.

Program 14

Question

- Write a program to count the number of vowels in a string (No control flow allowed).

Program

```
s = "Python Programming Is Fun"
text = s.lower()

# No if / for / while: use count() for each vowel and add the results
vowels = text.count("a") + text.count("e") + text.count("i") + text.count("o") + text.count("u")
print("String :", s)
print("Vowels :", vowels)

# Another way without any control flow statement
print("Vowels (map + sum):", sum(map(text.count, "aeiou")))
```

Output

```
String : Python Programming Is Fun
Vowels : 6
Vowels (map + sum): 6
```

i Remember

- **Logic:** *str.count()* counts how many times a character appears, so no loop or *if* is needed. *map()* applies count to each vowel.

Program 15

Question

- Write a program to check if a given key exists in a dictionary or not.

Program

```
student = {"name": "Asha", "age": 18, "city": "Kadapa"}

key = "age"
if key in student:                      # membership operator checks keys
    print(f"Key '{key}' exists, its value is {student[key]}")
else:
    print(f"Key '{key}' does not exist")

key = "phone"
print(f"Key '{key}' exists:", key in student)
print("Using get():", student.get("phone", "Key not found"))
```

Output

```
Key 'age' exists, its value is 18
Key 'phone' exists: False
Using get(): Key not found
```

Program 16

Question

- Write a program to add a new key-value pair to an existing dictionary.

Program

```
student = {"name": "Asha", "age": 18}
print("Original  :", student)

student["city"] = "Kadapa"                  # method 1: assignment
print("After add :", student)

student.update({"branch": "AI & ML"})       # method 2: update()
print("After update():", student)

student.setdefault("year", 2)               # method 3: setdefault()
print("After setdefault():", student)
```

Output

```
Original  : {'name': 'Asha', 'age': 18}
After add : {'name': 'Asha', 'age': 18, 'city': 'Kadapa'}
After update(): {'name': 'Asha', 'age': 18, 'city': 'Kadapa', 'branch': 'AI & ML'}
After setdefault(): {'name': 'Asha', 'age': 18, 'city': 'Kadapa', 'branch': 'AI & ML', 'year': 2}
```

Program 17

Question

- Write a program to sum all the items in a given dictionary.

Program

```
sales = {"Jan": 1200, "Feb": 1500, "Mar": 900, "Apr": 1800}

total = 0
for month in sales:                          # loop method
    total += sales[month]
print("Sum using loop    :", total)

print("Sum using sum()   :", sum(sales.values()))
print("Average           :", sum(sales.values()) / len(sales))
```

Output

```
Sum using loop    : 5400
Sum using sum()   : 5400
Average           : 1350.0
```

i Remember

- Use **d.values()** for the values. *sum(d)* would add the **keys**, not the values.

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. What is a dictionary? Can a list be a key? 2. Differentiate pop() and del for dictionaries. 3. What is the use of get()? 4. Differentiate list and tuple. 5. How do you create a one-item tuple? 6. What is zip()? 7. Differentiate remove() and discard(). 8. What is a frozenset?	1. Explain dictionary creation, access and modification with examples. 2. Explain any eight dictionary methods with examples. 3. Explain tuples with operations and built-in functions. 4. Explain the relation between tuples, lists and dictionaries. 5. Explain set methods and mathematical set operations with examples. 6. Compare list, tuple, set and dictionary. 7. Write programs 13–17 from the sample experiments.